

南开 Web 搜索引擎设计与实现报告

2412235_ 匡航逸

2026 年 6 月 2 日

目录

1	项目概述	4
2	作业要求对照	4
3	总体架构设计	6
4	爬虫系统设计	7
4.1	主题化抓取计划	7
4.2	Scrapy 工程结构	8
4.3	断点续爬与高价值队列	8
4.4	礼貌抓取与速度控制	8
5	数据清洗与文档解析	9
5.1	清洗目标	9
5.2	文档解析	9
6	索引构建	10
6.1	Elasticsearch Mapping	10
6.2	PageRank 计算	11
6.3	快照与联想词	11
7	查询解析	12
7.1	查询模式	12
7.2	高级查询语法	12
7.3	查询功能逐项说明	13
7.3.1	普通查询	13
7.3.2	站内查询	13
7.3.3	文档查询	14

目录	2
7.3.4 短语查询	14
7.3.5 通配查询	14
7.3.6 正则查询	15
7.3.7 查询日志	15
7.3.8 网页快照	15
8 检索与排序	16
8.1 Elasticsearch 查询构建	16
8.2 本地 BM25F 后端	16
9 Web 服务设计	17
9.1 FastAPI 应用	17
9.2 后端自动选择	17
9.3 单端口部署	17
10 前端界面设计	18
11 个性化与推荐	19
11.1 账号体系	19
11.2 个性化排序	20
11.3 搜索联想	20
12 运行与部署	21
12.1 依赖安装	21
12.2 启动服务	21
12.3 健康检查	21
13 测试与验证	22
13.1 自动化测试	22
13.2 前端构建验证	22
13.3 真实 ES 接口验证	22
14 文件与仓库管理	23
15 项目亮点	23
16 不足与改进方向	24
17 结论	24

目录	3
A 常用命令	24
A.1 启动	24
A.2 爬取	25
A.3 清洗	25
A.4 索引	25
A.5 测试	25
B 核心文件清单	26

1 项目概述

本项目围绕信息检索课程作业要求，实现了一个综合南开校内资源搜索引擎。系统面向南开大学主站、新闻站、院系站、教学招生、科研学术、图书馆、动漫资源和校园服务等多类站点进行抓取、清洗、索引和检索，并提供具备现代搜索引擎交互方式的 Web 控制台。系统并非只处理单一站点或少量样例数据，而是按 10 万页以上规模设计，真实本机索引验证达到 135779 条有效文档。

项目技术栈如下：

模块	技术选型
网页抓取	Scrapy、CrawlSpider、Rule、LinkExtractor、AutoThrottle、JOB DIR
数据存储	JSONL、SQLite、文件系统快照
索引引擎	Elasticsearch 8.15.1，本地 BM25F 兜底后端
Web 服务	FastAPI、Uvicorn、Cookie Session
前端界面	React、Vite、Ant Design
文档解析	pdfplumber、python-docx、openpyxl、文本清洗工具链
测试验证	pytest、FastAPI TestClient、前端 TypeScript 构建

系统主要目标包括：

1. 建立可扩展的南开校内资源爬取计划，覆盖多主题、多域名、多文件类型。
2. 构建包含标题、正文、URL、锚文本、文件类型、主题、PageRank、快照等字段的搜索索引。
3. 实现普通查询、站内查询、文档查询、短语查询、通配查询、正则查询、查询日志和网页快照。
4. 实现注册登录、兴趣词、历史查询、点击日志和个性化排序。
5. 实现搜索联想推荐和主题化 Web UI。
6. 保证项目可以在干净环境下启动、测试和演示，并通过 Git 忽略策略避免上传运行数据、虚拟环境和参考仓库。

2 作业要求对照

本节逐条对照课程要求，说明系统中对应的代码位置和完成情况。

要求	实现方式	状态
网页抓取	nku_search/crawl.py 调度 Scrapy; crawler_project/spiders/nku_crawl.py 实现 CrawlSpider; 按板块预算抓取南开主站、新闻、院系、教学、科研、图书、动漫和服务资源。	已完成
至少 10 万页	crawl_plan.py 默认计划 160000 页, 本机清洗后索引验证 135779 条有效记录。	已完成
文本索引	index.py 将 JSONL 转为 Elasticsearch 文档, 写入 title、text、url、anchors、filetype、section、category、pagerank 等字段。	已完成
标题、URL、锚文本	models.py 定义 CrawledPage; search_engine.py 定义 ES mapping; 查询时设置字段权重。	已完成
链接分析排序	ranking.py 实现 PageRank; index.py 构建图并写入 pagerank 字段; ES 使用 field_value_factor 融合排序。	已完成
站内查询	query.py 支持 site 参数和 site:domain 高级语法, 转换为 URL prefix filter。	已完成
文档查询	支持 PDF、DOC、DOCX、XLS、XLSX、PPT、PPTX、TXT 和 HTML 等类型的 filetype 过滤, 前端文档页可分页筛选。	已完成
短语查询	自动识别英文双引号和中文引号, 使用 ES multi_match type=phrase。	已完成
通配查询	支持 * 和 ?; ES 使用 wildcard 查询, 本地后端转换为正则。	已完成
正则查询	支持 /pattern/, ES 使用 regexp 查询。	扩展完成
查询日志	SQLite query_logs 保存查询、模式、站点、文件类型、结果数量和时间。	已完成
网页快照	索引阶段保存 HTML 或文档文本快照, /snapshot/{doc_id} 提供访问。	已完成
个性化查询	用户兴趣词和历史查询词作为排序 boost, 前端账号页支持编辑兴趣词。	已完成

要求	实现方式	状态
Web 界面	FastAPI 单端口托管 React 构建产物，Ant Design 控制台包含搜索、主题、文档、历史和账号页。	已完成
个性化推荐	/api/suggest 综合历史查询、索引标题词和用户个人历史生成搜索联想。	已完成

3 总体架构设计

系统采用分层架构，核心流程为：

```
Scrapy crawler
-> raw JSONL
-> clean / dedupe / normalize
-> PageRank / snapshots / suggestions
-> Elasticsearch index
-> FastAPI APIs + SQLite logs
-> React + Ant Design console
```

各层职责如下：

1. **爬虫层：**负责从南开校内多个站点抓取网页和附件，执行链接抽取、URL 规范化、主题分类、文档下载限制和断点续爬。
2. **清洗层：**过滤服务页、登录页、访问统计接口、空壳 HTML、异常状态码和重复 URL。
3. **索引层：**进行文本字段标准化、文档解析、PageRank 计算、快照保存、搜索联想词抽取和 ES 批量写入。
4. **检索层：**解析普通查询和高级查询语法，构建 Elasticsearch Query DSL，提供本地 BM25F 兜底。
5. **服务层：**FastAPI 提供 JSON API、会话、账号、日志、点击记录、快照和静态前端托管。
6. **展示层：**React + Ant Design 实现搜索框、联想、模式选择、分面筛选、分页、主题入口、文档页、历史页和账号页。

这样的分层使系统具备两个重要特性。第一，Elasticsearch 与本地后端解耦，即使 ES 未启动，仍可以通过内置 fixture 进行小规模测试。第二，前端与后端虽然在工程上分目录开发，但运行时由 FastAPI 统一托管，符合单端口搜索引擎演示需求。

4 爬虫系统设计

4.1 主题化抓取计划

爬虫计划由 `nku_search/crawl_plan.py` 管理。系统将南开资源划分为 8 个主题板块，默认总量为 160000 页，满足作业至少 100000 页要求，并预留扩容空间。

Section	主题	目标页数
news	新闻资讯	36000
main	学校门户	14000
schools	院系学科	42000
academic	教学招生	26000
research	科研学术	18000
library	图书资源	8000
anime	动漫资源	12000
services	校园服务	4000

其中动漫资源不只考虑单个页面，而是将 `12club.nankai.edu.cn` 及相关动漫主题资源纳入高价值 `frontier`。系统在抓取计划中保存每个板块的 `seed URL`、允许域名、深度限制、并发限制、礼貌延迟、文档优先策略和目标页数。

前端主题页直接读取 `/api/topics` 返回的索引统计，按上述主题展示每类文档数量，便于确认 10 万页规模下各板块覆盖情况，如图 1 所示。



图 1: 主题页面。系统按新闻、主站、院系、教学、科研、图书、动漫和校园服务展示已索引数量。

4.2 Scrapy 工程结构

爬虫相关代码集中在 `nku_search/crawler_project/`:

- `settings.py`: Scrapy 基础设置, 包括 robots、AutoThrottle、HTTP cache、请求指纹、日志等级和中间件。
- `spiders/nku_crawl.py`: 核心 CrawlSpider, 负责解析页面、抽取链接、识别文档、生成 Item。
- `middlewares.py`: URL 与响应头过滤, 控制大文件、无效 URL、服务接口和下载上限。
- `pipelines.py`: 板块预算、全局数量限制和 JSONL 写入。
- `logformatter.py`: 减少 Scrapy 在关闭时打印超长 item 的噪声。

4.3 断点续爬与高价值队列

Scrapy 的 JOBDIR 用于保存 scheduler 状态、dupefilter 指纹和未完成队列。为了避免旧低价值队列污染新策略, 项目在 `high_value_jobdir_for()` 中根据输出文件生成高价值队列目录:

```
data/crawl/jobs/<output-stem>-highvalue-frontier
```

这样既可以续爬同一轮任务, 又能在策略重构后使用新的 jobdir 避免继续消耗旧队列。

4.4 礼貌抓取与速度控制

系统默认设置较高的异步并发, 但仍保留 robots、AutoThrottle 和下载大小限制:

- `CONCURRENT_REQUESTS=128`
- `CONCURRENT_REQUESTS_PER_DOMAIN=8`
- `DOWNLOAD_DELAY=0.05`
- `DOWNLOAD_TIMEOUT=12`
- `DOWNLOAD_MAXSIZE=16MB`
- `NKU_MAX_DOCUMENT_DOWNLOAD_BYTES=3MB`
- `NKU_MAX_DOCUMENT_PARSE_BYTES=1MB`

Scrapy 本身基于 Twisted 异步 IO，不是每个请求创建一个操作系统线程。全局并发和单域名并发共同决定吞吐量。实际速度还受以下因素影响：

1. 南开不同子域名的服务器响应速度不同。
2. 大量 301/302、403、404、502、503 会消耗请求槽。
3. PDF、Word、Excel、PPT 等附件下载和解析成本高于 HTML。
4. robots.txt、DNS 失败和超时重试会降低有效 item 产出率。
5. 列表页、分页页和访问统计接口会产生大量低价值链接，因此清洗与 frontier 评分很关键。

5 数据清洗与文档解析

5.1 清洗目标

原始爬取结果中会包含重复 URL、服务接口、访问统计接口、空壳页面、异常状态页和大附件。清洗模块 `nku_search/clean.py` 的目标是将原始 JSONL 转换为可索引的高质量文档集合。

主要过滤规则：

- URL 为空或格式非法。
- 非 HTTP/HTTPS 协议。
- `_visitcount`、`/api/tracking`、`/wp-admin` 等服务路径。
- 登录、登出、后台、搜索接口等低价值路径。
- 状态码不在 `{200, 202}` 中。
- HTML 页面无标题、无正文、无 HTML 内容。
- 重复 URL，仅保留质量评分最高的记录。

5.2 文档解析

系统支持多种附件类型：

文件类型	处理方式
PDF	使用 pdfplumber 抽取文本
DOCX	使用 python-docx 抽取段落与表格文本
XLSX	使用 openpyxl 抽取工作表文本
DOC/XLS/PPT/PPTX	下载后保存元信息和可解析文本，标题通过 URL 与正文 fallback 归一化
HTML	BeautifulSoup 清理脚本、样式、导航噪声并抽取标题、正文和链接

文档标题乱码问题通过 `normalize_document_title()` 处理。对于 PDF、PPT、PPTX 等附件，系统优先使用可读标题、URL 文件名和正文前缀，而不是直接沿用乱码标题字段。

6 索引构建

6.1 Elasticsearch Mapping

Elasticsearch 索引定义位于 `nku_search/search_engine.py`。主要字段如下：

字段	类型	说明
<code>doc_id</code>	keyword	文档稳定 ID，由 URL 等信息生成
<code>url</code>	keyword	原始 URL，用于站内过滤和结果展示
<code>title</code>	text + keyword	标题，text 用于检索，keyword 用于通配
<code>text</code>	text	正文或文档文本
<code>anchors</code>	text	指向该页面或页面中抽取的锚文本
<code>outgoing_links</code>	keyword	页面外链，用于 PageRank 图和分析
<code>content_type</code>	keyword	HTTP Content-Type
<code>filetype</code>	keyword	html/pdf/docx/xlsx 等类型
<code>section</code>	keyword	内部板块，如 news、schools、anime
<code>category</code>	keyword	中文主题，如新闻资讯、院系学科
<code>fetches_at</code>	date	抓取时间
<code>status</code>	integer	HTTP 状态码

字段	类型	说明
pagerank	float	链接分析得分
snapshot_path	keyword	快照文件名

索引设置中将 `max_result_window` 提高到 200000，以支持 10 万级结果分页。中文分词当前采用 Elasticsearch standard tokenizer 加 lowercase filter，保证部署简单和可复现。

6.2 PageRank 计算

链接图构建逻辑位于 `index.py`：

1. 第一遍扫描 JSONL，建立 URL 到 `doc_id` 的映射。
2. 第二遍读取 `outgoing_links`，将站内链接转换为 `doc_id` 边。
3. 对每个页面限制最多 64 条有效出边，避免导航页面边数过大。
4. 使用 `compute_pagerank()` 迭代计算得分。
5. 将 `pagerank` 写入 Elasticsearch 文档字段。

PageRank 公式采用经典随机游走模型：

$$PR(v) = \frac{1-d}{N} + d \sum_{u \in In(v)} \frac{PR(u)}{L(u)}$$

其中 d 为阻尼系数， N 为图中节点数， $In(v)$ 为指向页面 v 的页面集合， $L(u)$ 为页面 u 的出链数量。

6.3 快照与联想词

索引阶段会将每个文档保存为快照：

```
data/snapshots/<doc_id>.html
```

HTML 页面保存原始 HTML；附件保存包含标题和文本的简化 HTML。这样当原始网页变化、删除或访问失败时，用户仍可通过搜索结果中的“快照”查看抓取时的内容。

联想词由 `recommend.py` 从标题和正文中抽取，写入 SQLite `suggestions` 表。用户输入时调用 `/api/suggest` 获取候选词。

7 查询解析

7.1 查询模式

查询模式由 `nku_search/query.py` 中的 `detect_mode()` 判断：

- 显式模式优先，例如前端选择 `phrase`、`wildcard`、`regex`。
- `/.../` 自动识别为正则查询。
- 查询中出现 `*` 或 `?` 自动识别为通配查询，但会忽略高级语法中 URL 参数里的通配符。
- 英文双引号和中文引号包围的内容识别为短语查询。
- 其他情况为普通查询。

短语查询和普通查询的差异如下：

查询	含义
南开大学召开	普通查询，将输入拆成关键词，结果只要相关即可命中，词语可以不连续。
“南开大学召开”	短语查询，要求“南开大学召开”作为连续短语出现，顺序和相邻关系更严格。

7.2 高级查询语法

`advanced_query.py` 支持以下语法：

- `site:news.nankai.edu.cn` 南开
- `filetype:pdf` CARSI
- `title: 人工智能南开`
- `inurl:graduate` 招生
- `section:anime` 动漫
- `category: 新闻资讯南开`
- `after:2026-01-01 before:2026-06-01` 南开
- `-词语` 或负向高级项，用于排除结果。

解析器会把高级语法转换为结构化字段，例如 `site`、`filetype`、`title_terms`、`url_terms`、`excluded_terms`、`phrases`、`wildcard_patterns` 和 `regex_patterns`。这样可以避免直接用字符串拼接查询 DSL。

7.3 查询功能逐项说明

作业要求中的查询能力包括站内查询、文档查询、短语查询、通配查询、查询日志和网页快照。本系统在此基础上补充普通查询、正则查询、标题查询、URL 查询、日期过滤、主题过滤和板块过滤。各类查询均由 `nku_search/query.py` 与 `nku_search/advanced_query.py` 解析，最终通过 Elasticsearch Query DSL 或本地 BM25F 后端执行。

7.3.1 普通查询

普通查询是默认检索方式，适用于用户直接输入关键词的场景。例如：

南开大学 召开

系统会将其识别为 NORMAL 模式，并在标题、锚文本、正文和 URL 中执行多字段相关性检索。普通查询关注关键词整体相关性，多个词可以分散出现在不同位置，适合召回范围较大的综合搜索。

普通查询的实际界面如图 2 所示。示例词为 南开大学召开，页面展示相关结果、分面统计和分页入口。



图 2: 普通查询示例。查询词为 南开大学召开，系统按关键词相关性召回结果。

7.3.2 站内查询

站内查询用于限定结果来源站点，既可以通过前端筛选项提交，也可以直接在搜索框输入高级语法。例如：

```
site:news.nankai.edu.cn 南开
site:https://news.nankai.edu.cn/ 南开
```

`site_prefix_filters()` 会将域名形式扩展为 `https://` 和 `http://` 两类 URL prefix filter。这样能够兼容南开不同站点同时使用 HTTP 与 HTTPS 的实际情况。

7.3.3 文档查询

文档查询用于搜索 PDF、Word、Excel、PPT、TXT 和 HTML 等文件类型。典型输入如下：

```
filetype:pdf CARSI
filetype:ppt 招生
```

索引阶段会保存文档的 `filetype` 字段；查询阶段将 `filetype` 转换为 term filter。前端文档页使用同一接口，并提供分页、文件类型筛选和结果卡片中的文档入口。

7.3.4 短语查询

短语查询用于要求词语按固定顺序连续出现。系统同时支持英文双引号和中文引号。例如：

```
"南开大学召开"
“南开大学召开”
```

后端会将短语查询识别为 PHRASE 模式，并使用 `multi_match type=phrase`。与普通查询 南开大学召开相比，短语查询约束更强，结果数量更少，但命中内容更贴近精确表述。

短语查询界面如图 3 所示。示例词为“南开大学召开”，用于展示连续短语匹配与普通关键词匹配的差异。



图 3: 短语查询示例。查询词为“南开大学召开”，系统要求词语按照固定顺序连续出现。

7.3.5 通配查询

通配查询支持 * 和 ? 两类符号。* 表示任意长度字符，? 表示单个字符。例如：

```
南开*
计?
```

Elasticsearch 后端会在 `title.keyword`、`text` 和 `anchors` 字段上构建 wildcard 查询；本地后端会把通配表达式转换为正则表达式后匹配。系统会过滤 URL 参数中的无意义通配符，避免把统计接口或分页参数误识别为用户查询。

7.3.6 正则查询

正则查询是扩展能力，用于表达更灵活的模式匹配。输入形式为：

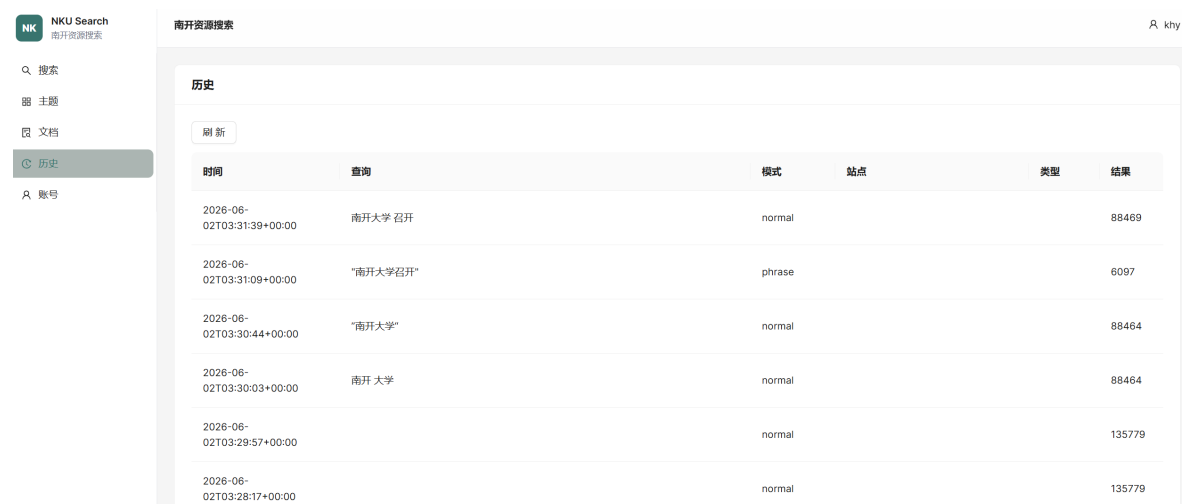
```
/南.*/
```

后端会将其识别为 REGEX 模式，并在 Elasticsearch 中使用 regexp 查询。本地后端使用 Python 正则进行匹配。该能力可以覆盖更复杂的标题、正文和专有名词模式检索。

7.3.7 查询日志

每次调用 `/api/search` 后，系统都会将查询词、查询模式、站点筛选、文件类型、结果数量、用户编号和时间写入 SQLite 的 `query_logs` 表。`/api/history` 读取该表，前端历史页面展示查询记录。日志既用于作业要求中的查询记录演示，也作为搜索联想和个性化排序的数据来源。

查询历史页面如图 4 所示，能够直接查看每次检索的关键词、模式、过滤条件、结果数量和时间。



时间	查询	模式	站点	类型	结果
2026-06-02T03:31:39+00:00	南开大学 召开	normal			88469
2026-06-02T03:31:09+00:00	"南开大学召开"	phrase			6097
2026-06-02T03:30:44+00:00	"南开大学"	normal			88464
2026-06-02T03:30:03+00:00	南开 大学	normal			88464
2026-06-02T03:29:57+00:00		normal			135779
2026-06-02T03:28:17+00:00		normal			135779

图 4: 查询历史页面。SQLite 保存查询词、模式、站点、文件类型、结果数量和时间。

7.3.8 网页快照

网页快照在索引阶段生成。`save_snapshot()` 会将网页 HTML 或文档文本保存到 `data/snapshots/<doc_id>.html`。搜索结果卡片提供快照入口，路由 `/snapshot/<doc_id>` 会读取对应文件并返回给浏览器。即使原网页之后发生变化或暂时不可访问，系统仍可展示索引时保存的内容。

8 检索与排序

8.1 Elasticsearch 查询构建

普通查询使用 `multi_match best_fields`:

```
fields = ["title^3", "anchors^2", "text", "url"]
```

短语查询使用:

```
multi_match type=phrase  
fields = ["title^3", "anchors^2", "text"]
```

通配查询使用 `title keyword`、`text`、`anchors` 的 `wildcard` 子句; 正则查询使用 `title keyword` 和 `text` 的 `regexp` 子句。站内查询会被转换为 `URL prefix filter`。域名形式 `news.nankai.edu.cn` 会同时生成 `https://` 和 `http://` 两个 `prefix`, 以兼容不同站点协议。

最终排序使用 `function_score`:

```
text relevance + field_value_factor(pagerank) + user interest should clauses
```

其中 `pagerank` 的配置为:

```
field = "pagerank"  
factor = 0.15  
modifier = "ln1p"  
missing = 0
```

8.2 本地 BM25F 后端

本地后端用于测试、诊断和 ES 不可用时的兜底演示。它维护:

- vocabulary
- inverted index
- document frequency
- field length
- title/text/anchors/url 字段权重
- facets
- diagnostics

BM25F 将多个字段合并计分。标题和锚文本权重更高, 正文提供主要召回, URL 命中用于补充站点和路径相关性。

9 Web 服务设计

9.1 FastAPI 应用

FastAPI 应用位于 `nku_search/web.py`。核心接口如下：

接口	说明
GET <code>/api/search</code>	搜索接口，支持 <code>q</code> 、 <code>site</code> 、 <code>filetype</code> 、 <code>section</code> 、 <code>category</code> 、 <code>mode</code> 、 <code>page</code> 、 <code>size</code> 。
GET <code>/api/suggest</code>	搜索联想接口。
GET <code>/api/topics</code>	主题板块及每类索引数量。
GET <code>/api/stats</code>	后端、文档数、特征、 <code>schema</code> 和统计信息。
GET <code>/api/history</code>	查询日志。
POST <code>/api/register</code>	用户注册。
POST <code>/api/login</code>	用户登录并写入 <code>session cookie</code> 。
POST <code>/api/profile</code>	更新用户兴趣词。
POST <code>/api/click</code>	记录搜索结果点击。
GET <code>/snapshot/{doc_id}</code>	返回网页快照。
GET <code>/health</code>	健康检查，返回后端类型和索引文档数。

9.2 后端自动选择

系统启动时优先连接 `Elasticsearch`。如果 `ES` 可用、索引存在且文档数大于 0，则使用 `SearchBackend`。如果 `ES` 不可用，则加载 `nku_search/fixtures/sample_documents.jsonl` 并使用 `LocalSearchBackend`。

此外，`BackendManager` 会在请求时重新检查 `ES`。当服务启动早于 `Elasticsearch`、初始落到 `Local` 后端时，只要后续 `ES` 恢复并且索引可用，系统会自动切回 `SearchBackend`，避免用户看到 0 结果或 6 条样例数据。

9.3 单端口部署

前端源码位于 `frontend/`，构建产物位于 `frontend/dist/`。运行时不再启动独立 `Vite dev server`，而是由 `FastAPI` 挂载：

- `/assets`: `React` 静态资源。

- `/`: React SPA 入口。
- `/full_path:path`: 前端路由兜底。

启动器 `nku_search/serve.py` 会检查前端构建是否过期。如果 `frontend/src/package.json`、`vite.config.ts` 等文件比 `frontend/dist/index.html` 新，则自动执行：

```
npm run build
```

这样可以避免后端启动后仍加载旧前端 bundle。

10 前端界面设计

前端主组件为 `frontend/src/SearchConsole.tsx`。界面包含五个页面：

1. **搜索**：主搜索框、推荐词、模式选择、分面筛选、结果列表、上下分页、快照入口。
2. **主题**：显示八个主题板块，每个板块展示中文主题、说明和已索引数量。
3. **文档**：按文件类型切换 PDF、DOC、DOCX、XLS、XLSX、PPT、PPTX、TXT、HTML。
4. **历史**：展示查询历史、模式、站点、文件类型、结果数量和时间。
5. **账号**：注册、登录、兴趣词编辑和退出登录。

前端交互遵循现代搜索 UI 逻辑：

- 搜索框默认为空，页面加载时展示全库结果和分面统计。
- 推荐词点击后同步到搜索框并立即搜索。
- 站点、类型、主题作为左侧 facet，而不是额外堆叠多个搜索栏。
- 分页会保留当前查询和筛选条件。
- 文档页和综合搜索页都支持分页。
- 快照按钮与原始链接分离，方便演示网页备份功能。

综合搜索首页如图 5 所示。截图来自 FastAPI 单端口服务 `http://127.0.0.1:8000`，前端静态资源与后端 API 均由同一服务提供。

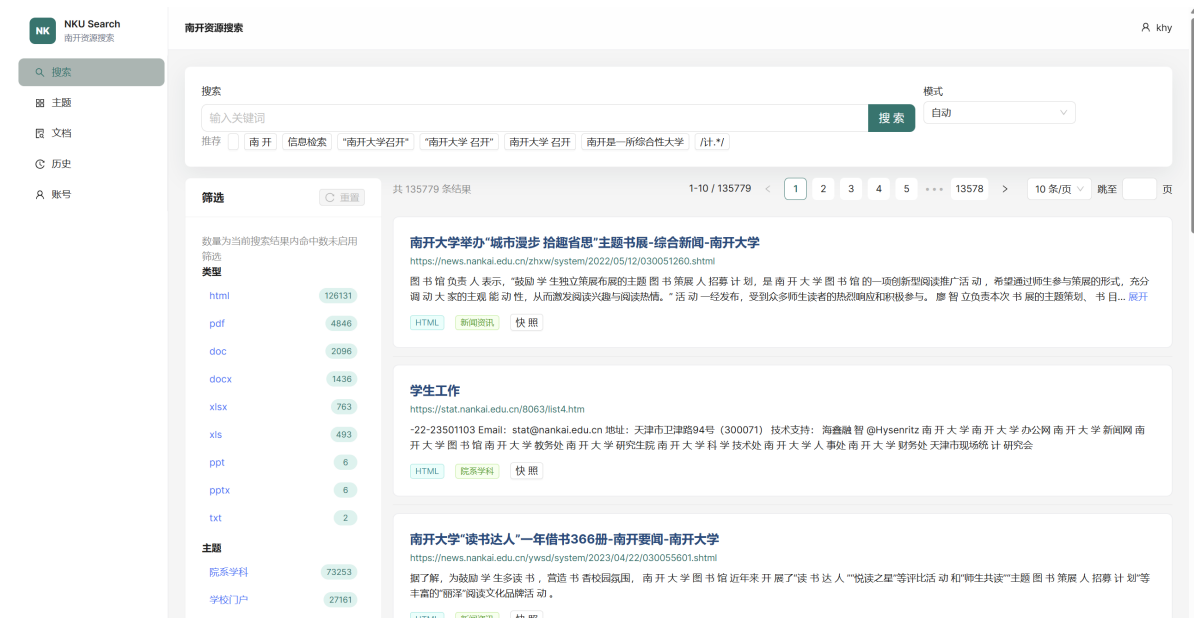


图 5: 综合搜索首页。搜索框为空时展示全库结果、分面筛选、分页和推荐词入口。

11 个性化与推荐

11.1 账号体系

账号系统由 `auth.py` 和 `storage.py` 实现。SQLite 表包括：

- `users`：用户名、密码 hash、兴趣词。
- `sessions`：session token 和用户 ID。
- `query_logs`：查询日志。
- `click_logs`：点击日志。
- `suggestions`：搜索联想词。

密码使用 `passlib bcrypt hash`，不明文保存。登录后 `FastAPI` 写入 `HttpOnly cookie`，后续请求根据 `cookie` 获取当前用户。

账号页面如图 6 所示，用户可以注册、登录并维护兴趣词，兴趣词会进入个性化排序逻辑。



图 6: 账号页面。用户可以注册、登录、更新兴趣词，兴趣词会参与个性化排序。

11.2 个性化排序

个性化排序的输入包括：

1. 用户兴趣词，例如“人工智能信息检索图书馆”。
2. 用户历史查询词。
3. 当前查询词。

ES 后端将用户兴趣词作为 `should` 子句加入查询：

```
match text with boost = 0.4
```

本地后端在 `feature vector` 中加入 `personal` 分项。这样登录用户与游客在同一查询下可能看到不同排序。

11.3 搜索联想

搜索联想来源包括：

- 索引阶段从标题和正文抽取的高频词。
- 用户历史查询。
- 全局热门查询。

用户在搜索框输入时，前端调用：

```
GET /api/suggest?q=<prefix>
```

接口返回最多 10 条建议。点击建议会同步搜索框内容并发起查询。

搜索联想交互如图 7 所示。推荐词来自索引词表、全局历史和用户个人历史，点击后会同步搜索框并发起检索。

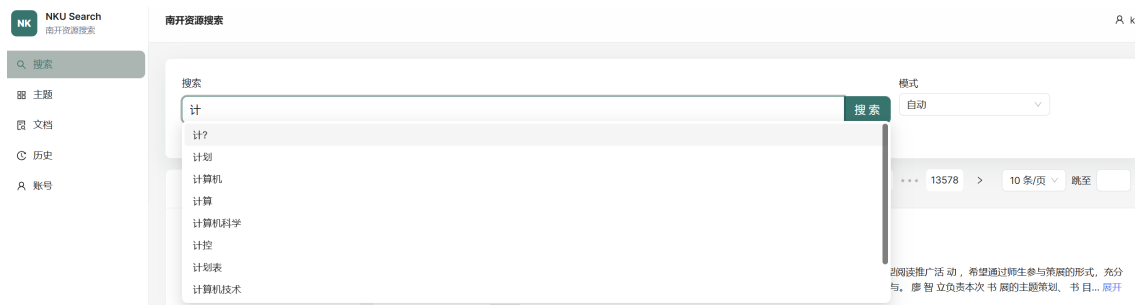


图 7: 搜索联想推荐。输入或点击推荐词后，前端会同步搜索框并触发查询。

12 运行与部署

12.1 依赖安装

```
cd /d D:\hw4
python -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend
npm.cmd install --cache ..\.npm-cache
cd ..
```

12.2 启动服务

```
docker compose up -d elasticsearch
.\venv\Scripts\python.exe -B -m nku_search.serve --host 127.0.0.1 --port 8000
```

浏览器访问:

```
http://127.0.0.1:8000
```

12.3 健康检查

```
curl http://127.0.0.1:8000/health
```

真实索引可用时返回类似:

```
{"status": "ok", "backend": "SearchBackend", "elasticsearch": true, "index": "nku_pages", "documents": 135779}
```

如果返回 LocalSearchBackend, 说明 ES 未启动、索引为空、9200 不可访问, 或 8000 上运行的是旧进程。

13 测试与验证

13.1 自动化测试

项目使用 pytest 覆盖主要模块：

- 查询解析：短语、通配、正则、高级语法。
- 排序模型：BM25F、PageRank、个性化兴趣分数。
- 文本处理：HTML 清洗、链接抽取、文档标题归一化。
- 爬虫结构：Scrapy settings、middleware、pipeline、spider 规则。
- 数据清洗：重复 URL、服务 URL、异常状态过滤。
- 索引流程：PageRank 图、快照、ES 文档生成。
- Web API：搜索、账号、历史、主题、快照。
- SQLite：用户、会话、日志和联想词存储。

测试命令：

```
..\venv\Scripts\python.exe -B -m pytest
```

当前结果：

```
67 passed, 3 warnings
```

13.2 前端构建验证

```
cd frontend
npm.cmd run build
cd ..
```

构建通过后生成 frontend/dist/，但该目录不提交到 Git。服务启动器会在需要时自动构建。

13.3 真实 ES 接口验证

当前本机真实 Elasticsearch 索引验证数据：

验证项	结果
索引文档数	135779
普通搜索 南开大学召开	88469
短语搜索 “南开大学召开”	6097
站内搜索 <code>site:news.nankai.edu.cn</code> 南开	5723
PDF 文档	4846
动漫主题	1516

14 文件与仓库管理

Git 仓库只提交源码、测试、文档、配置和小型 fixture。真实运行数据不提交，包括：

- `data/`
- `elasticsearch-data/`
- `references/`
- `frontend/dist/`
- `frontend/node_modules/`
- `.venv/`
- `.npm-cache/`
- `.uv-cache/`

报告目录采用特殊策略：Git 中只保留最终 PDF 和 `report/fig/` 下的图片；LaTeX 源码和编译中间文件不提交。

15 项目亮点

1. **完整链路：**从爬虫、清洗、索引、排序、API 到前端界面全部打通。
2. **大规模设计：**默认计划 16 万页，真实索引验证超过 13 万条。
3. **多文件类型：**覆盖 HTML、PDF、DOC、DOCX、XLS、XLSX、PPT、PPTX。
4. **高级查询能力：**支持站内、文档、短语、通配、正则、标题、URL、日期过滤。

5. **排序融合**: 结合文本相关性、PageRank 和用户兴趣。
6. **个性化功能**: 账号体系、兴趣词、历史日志、点击日志、搜索联想。
7. **单端口部署**: FastAPI 统一托管 React 前端和 JSON API, 演示简单。
8. **工程质量**: 测试覆盖 67 个用例, 包含诊断、fixture 和 ES 兜底机制。

16 不足与改进方向

当前系统已经满足课程要求, 但仍有可继续优化的方向:

1. 中文分词可替换为 IK Analyzer 或自定义词典, 以提升中文短语和专有名词召回。
2. 可加入 Learning to Rank 或基于点击日志的排序模型, 使个性化排序更强。
3. 可将爬虫 frontier 持久化到独立数据库, 支持更细粒度的任务调度和失败恢复。
4. 可对 PDF、PPT、DOC 等文档增加 OCR 和更强的元数据解析。
5. 可加入后台管理页, 展示爬虫进度、索引状态、失败 URL 和数据质量。
6. 可使用 Docker Compose 同时编排 FastAPI 和前端构建, 降低部署步骤。

17 结论

本项目实现了一个面向南开校内资源的综合 Web 搜索引擎, 覆盖网页抓取、文本索引、查询服务、个性化查询、Web 界面和搜索推荐等主要模块。系统使用 Scrapy 进行工程化大规模抓取, 使用 Elasticsearch 构建可分页、可过滤、可高亮的搜索服务, 使用 FastAPI 和 SQLite 管理查询、用户和日志, 使用 React + Ant Design 提供可实际操作的搜索界面。

从功能覆盖角度看, 系统完成了作业要求的六类查询功能, 包括站内查询、文档查询、短语查询、通配查询、查询日志和网页快照; 同时扩展了正则查询、标题查询、URL 查询、主题筛选、个性化排序和搜索联想。从工程实现角度看, 系统具备断点续爬、数据清洗、PageRank、快照保存、ES 自动切换、本地兜底、自动前端构建和完整测试, 能够支撑课程演示和后续扩展。

A 常用命令

A.1 启动


```
cd /d D:\hw4
docker compose up -d elasticsearch
.\.venv\Scripts\python.exe -B -m nku_search.serve --host 127.0.0.1 --port 8000
```

A.2 爬取

```
.\.venv\Scripts\python.exe -B -m nku_search.crawl --list-sections
.\.venv\Scripts\python.exe -B -m nku_search.crawl --max-pages 160000 --output
data/crawl/pages_160k.jsonl
```

A.3 清洗

```
.\.venv\Scripts\python.exe -B -m nku_search.clean --input data/crawl/pages_160k.
jsonl --output data/crawl/pages_160k_clean.jsonl
```

A.4 索引

```
.\.venv\Scripts\python.exe -B -m nku_search.index --input data/crawl/
pages_160k_clean.jsonl --reset
```

A.5 测试

```
.\.venv\Scripts\python.exe -B -m pytest
cd frontend
npm.cmd run build
cd ..
```

B 核心文件清单

文件	作用
<code>nku_search/crawl.py</code>	爬虫 CLI、Scrapy 设置、frontier 扩展和 JOBDIR 管理。
<code>nku_search/crawl_plan.py</code>	主题板块、种子 URL、目标页数和允许域名。
<code>nku_search/crawler_project/spiders/nku_crawl.py</code>	核心 CrawlSpider，负责页面解析、文档识别、链接抽取和 Item 生成。
<code>nku_search/clean.py</code>	JSONL 清洗、去重、服务 URL 过滤和质量筛选。
<code>nku_search/index.py</code>	PageRank 计算、快照保存、联想词抽取和 Elasticsearch 批量索引。
<code>nku_search/search_engine.py</code>	Elasticsearch 后端、本地后端入口、索引 mapping 和搜索结果转换。
<code>nku_search/query.py</code>	查询模式识别、站内过滤、短语/通配/正则查询和 Query DSL 构建。
<code>nku_search/advanced_query.py</code>	高级查询语法解析，包括 site、filetype、title、inurl、日期和排除词。
<code>nku_search/local_index.py</code>	本地 BM25F 倒排索引、facet、诊断信息和兜底检索。
<code>nku_search/web.py</code>	FastAPI API、账号、日志、快照、健康检查和静态前端托管。
<code>frontend/src/SearchConsole.tsx</code>	React + Ant Design 主界面，包含搜索、主题、文档、历史和账号页。
<code>tests/</code>	自动化测试，覆盖查询、爬虫、索引、Web API、存储和文本处理。